

MongoDB Shell[Install mongosh](#)[Connect to a Deployment](#)[► Configure mongosh](#)[Run Commands](#)[► Perform CRUD Operations](#)[Run Aggregation Pipelines](#)[Client-Side Field Level
Encryption](#)[► Write Scripts](#)[► Snippets](#)[▼ Reference](#)[Compatibility Changes with
Legacy mongo Shell](#)[Data Types](#)[Methods](#)[▼ EJSON](#)[EJSON.parse\(\)](#)**EJSON.stringify()**[.mongoshrc Configuration
File](#)[Options](#)[Docs Home](#) → [View & Analyze Data](#) → [MongoDB Shell](#)

EJSON.stringify() [🔗](#)

On this page

Syntax

[Command Fields](#)[Behavior](#)[Examples](#)[Change Output Spacing](#)[Select Output Fields](#)[Use a Function to Transform Output Fields](#)[Write to a File from Inside mongosh](#)[Run from the Command Line](#)[Filter Output Fields](#)[Legacy tojsononeline\(\)](#)[Learn More](#)

The `EJSON.stringify()` method converts BSON values to strings.

Syntax

The `EJSON.stringify()` method takes a BSON object as input and optional modifiers that control the format of the output string.

On this page

Syntax

[Command Fields](#)[Behavior](#)[Examples](#)[Change Output
Spacing](#)[Select Output Fields](#)[Use a Function to
Transform Output
Fields](#)[Write to a File from
Inside mongosh](#)[Run from the
Command Line](#)[Filter Output Fields](#)[Legacy
tojsononeline\(\)](#)[Learn More](#)

```
EJSON.stringify(BSON object, [replacer], [space], [options])
```

Command Fields

The `EJSON.stringify()` method has these fields:

Field	Type	Necessity	Description
value	BSON object	Required	Object <code>EJSON.stringify()</code> transforms
replacer	array or function	Optional	Modifies output. If the value exists but it is not an array or a function, <code>EJSON.stringify()</code> returns all document fields.
		Value	Effect
		array	An array of document fields to include in the output
		function	A function that takes two parameters, <code>key</code> and <code>value</code> . <code>key</code> provides the function's <code>this</code> context. <code>EJSON</code> returns the transformed <code>value</code> . See below for an example

Field	Type	Necessity	Description						
<code>spacer</code>	integer or string	Optional	Controls spacing in the output. Use <code>null</code> as a place holder for <code>replacer</code> if you only want to specify the <code>spacer</code> option.						
			<table><tr><th>Value</th><th>Effect</th></tr><tr><td>integer</td><td>The number of spaces to indent each level. <code>10</code> is the maximum.</td></tr><tr><td>string</td><td>A character to use to indent each level. This option produces invalid JSON if you use a character other than a space or tab. For more information see <code>JSON.stringify()</code> ↗</td></tr></table>	Value	Effect	integer	The number of spaces to indent each level. <code>10</code> is the maximum.	string	A character to use to indent each level. This option produces invalid JSON if you use a character other than a space or tab. For more information see <code>JSON.stringify()</code> ↗
Value	Effect								
integer	The number of spaces to indent each level. <code>10</code> is the maximum.								
string	A character to use to indent each level. This option produces invalid JSON if you use a character other than a space or tab. For more information see <code>JSON.stringify()</code> ↗								
<code>options</code>	boolean	Optional	Additional configuration options						
			<table><tr><th>Option</th><th>Default</th><th>Meaning</th></tr><tr><td><code>relaxed</code></td><td><code>true</code></td><td>Enable relaxed mode for Extended JSON. Returns native JSON types instead of attaching BSON type information where applicable.</td></tr></table>	Option	Default	Meaning	<code>relaxed</code>	<code>true</code>	Enable relaxed mode for Extended JSON. Returns native JSON types instead of attaching BSON type information where applicable.
Option	Default	Meaning							
<code>relaxed</code>	<code>true</code>	Enable relaxed mode for Extended JSON. Returns native JSON types instead of attaching BSON type information where applicable.							

Behavior

You can call the EJSON interface from inside an interactive `mongosh` session or from the system command line using `--eval`.

Call the EJSON interface from an interactive session:

```
EJSON.stringify( db.sales.find().toArray(), null, 2 )
```



Call the EJSON interface from the system command line:

```
mongosh --eval "EJSON.stringify( db.sales.find().toArray(), null, 2)
```



To control how documents are passed to EJSON, use one of the `mongosh` cursor method iterators.

Iterator	Characteristics
<code>.toArray()</code>	Blocking, buffers the entire result
<code>.foreach</code>	Non-blocking, prints documents one-by-one
<code>.next()</code>	Non-blocking, manually iterate over results

Examples

To try these examples, first create the `sales` collection:

```
db.sales.insertMany( [  
  { custId: 345, purchaseDate: ISODate("2023-07-04"), quantity: 4,  
    { custId: 346, purchaseDate: ISODate("2023-07-12"), quantity: 3,  
    { custId: 486, purchaseDate: ISODate("2023-08-01"), quantity: 9,  
  ] )
```



Change Output Spacing

To increase indentation between levels, set the `spacing` option.

```
EJSON.stringify( db.sales.findOne( { custId: 345 } ), null , 5 )
```



`EJSON.stringify()` indents each document level five spaces.

```
{
  "_id": {
    "$oid": "64da90c1175f5091debcab26"
  },
  "custId": 345,
  "purchaseDate": {
    "$date": "2023-07-04T00:00:00Z"
  },
  "quantity": 4,
  "cost": {
    "$numberDecimal": "100.60"
  }
}
```



Select Output Fields

To select a subset of document fields, use an array to set the `replace` option.

```
EJSON.stringify( db.sales.find().toArray(), [ "quantity", "cost" ] )
```



EJSON formats the `quantity` and `cost` for each document.

```
[{"quantity":4,"cost":{}},{ "quantity":3,"cost":{}},{ "quantity":9,"co
```



The `spacing` option is not specified in this example, so EJSON returns the selected fields on a single line.

Use a Function to Transform Output Fields

To transform field values, use a JavaScript function to set the `replace` option.

```
let queryResults = db.sales.find().toArray()
let replacer = function( key, value ){
  if ( key === '_id' ) {
    value = undefined;
  }
  if ( key === 'quantity' ) {
    value = 2 * value;
  }

  return value;
}

EJSON.stringify( queryResults, replacer, 3 )
```



The function runs recursively against the input object.

```
[
  {
    "custId": 345,
    "purchaseDate": {
      "$date": "2023-07-04T00:00:00Z"
    },
    "quantity": 8,
    "cost": {
      "$numberDecimal": "100.60"
    }
  },
  {
    "custId": 346,
    "purchaseDate": {
      "$date": "2023-07-12T00:00:00Z"
    },
    "quantity": 6,
    "cost": {
      "$numberDecimal": "175.45"
    }
  },
  {
    "custId": 486,
    "purchaseDate": {
      "$date": "2023-08-01T00:00:00Z"
    },
    "quantity": 18,
    "cost": {
      "$numberDecimal": "200.53"
    }
  }
]
```



The `replacer` function updates two fields, `_id` and `quantity`.

`EJSON.stringify()` ignores fields with undefined values, so setting `_id: undefined` removes the `_id` field from the output string.

The function also modifies the `quantity` field in the output string. All of the `quantity` values are multiplied by two in the output string. `EJSON.stringify()` does not update the collection.

Write to a File from Inside mongosh

To write to a file from within `mongosh`, use the `fs` API. Use `EJSON.stringify()` to format the string that you pass to `fs`.

```
const sales_2023_07 = db.sales.find(
  {
    purchaseDate:
      {
        $gte: ISODate( "2023-07-01" ),
        $lte: ISODate( "2023-07-31" )
      }
  }
)

fs.writeFileSync(
  'sales_2023_07.json',
  EJSON.stringify( sales_2023_07.toArray(), null, 2 )
)
```

The example queries the `sales` collection for sales in July, 2023.

- `sales_2023_07` stores a MongoDB BSON object.
- `EJSON.stringify()` formats the object as a JSON string.

- ## Run from the Command Line

```
# Note: This example is formatted to fit on the page.
```



The command returns a single line of JSON for each document:

- The output is:

The single-line output format is convenient for scripting. `EJSON.stringify()` can also produce human-readable formatting:

Note: This example is formatted to fit on the page.



```
mongosh --quiet \  
  --eval "db.sales.find().forEach( \  
    o => print( EJSON.stringify(o, null, 3 ) ) )"

```

The output is:

Note: This is only the first document.

```
{  
  "_id": {  
    "$oid": "64da90c1175f5091debcab26"  
  },  
  "custId": 345,  
  "purchaseDate": {  
    "$date": "2023-07-04T00:00:00Z"  
  },  
  "quantity": 4,  
  "cost": {  
    "$numberDecimal": "100.60"  
  }  
}
```

- `o` is the BSON value that `EJSON.stringify()` converts on each iteration of `.forEach()`.
- `null` is a place holder for an optional `replacer`. When the `replacer` is absent, `EJSON.stringify()` returns all fields that have a defined value.
- `3` is the `spacer` value. It tells `EJSON.stringify()` to indent each new level by 3 spaces.

If the you want the output string to have additional type information, add the `{ relaxed: false }` option:

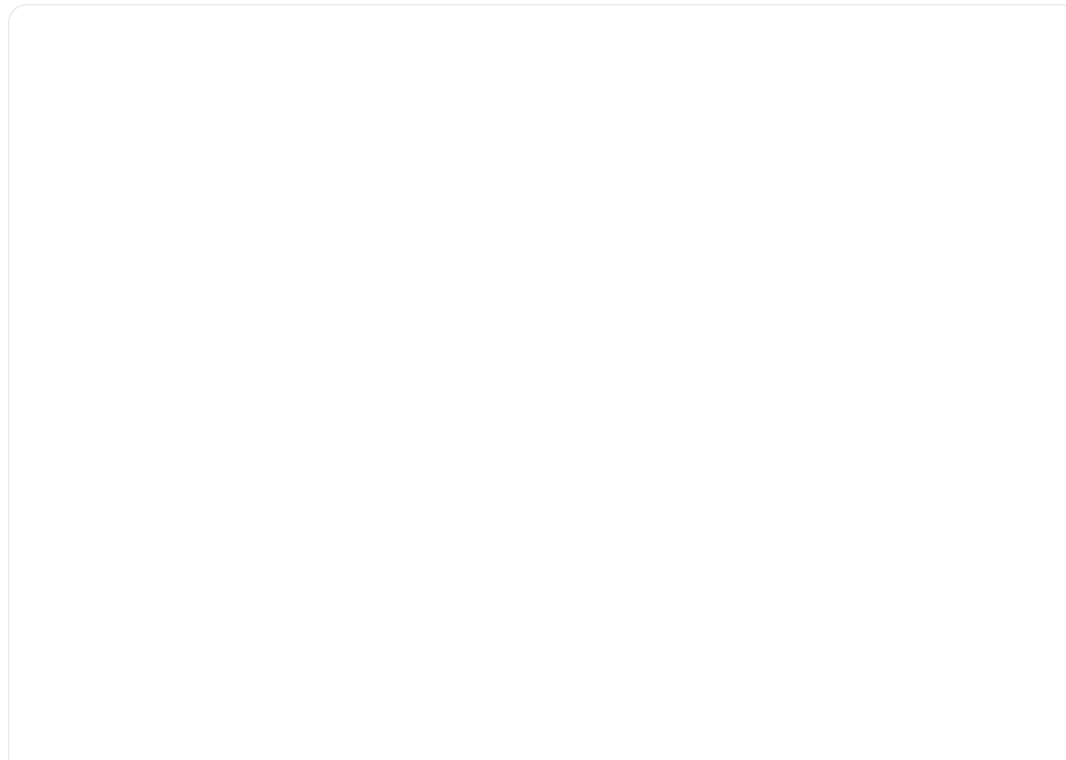
Note: This example is formatted to fit on the page.



```
mongosh --quiet \  
  --eval "db.sales.find().forEach( \  
    o => print( \  
      EJSON.stringify( o, null, 3, { relaxed: false } ) \  
    ) )"

```

The output is:



Note: This is only the first document.

```
{
  "_id": {
    "$oid": "64da90c1175f5091debcab26"
  },
  "custId": {
    "$numberInt": "345"
  },
  "purchaseDate": {
    "$date": {
      "$numberLong": "1688428800000"
    }
  },
  "quantity": {
    "$numberInt": "4"
  },
  "cost": {
    "$numberDecimal": "100.60"
  }
}
```

Filter Output Fields

`EJSON.stringify()` provides formatting options that reduce the need for an additional JSON parser like `jq`.

Note: This example is formatted to fit on the page.



```
mongosh --quiet \  
  --eval "EJSON.stringify( \  
    db.sales.find( {}, \  
      { _id: 0, custId: 1, quantity: 1 } ).toArray(), null \  
    );"
```

The output is:

```
[  
  {  
    "custId": 345,  
    "quantity": 4  
  },  
  {  
    "custId": 346,  
    "quantity": 3  
  },  
  {  
    "custId": 486,  
    "quantity": 9  
  }  
]
```

Legacy `tojsononeline()`

`mongosh` formats query output differently than the deprecated `mongo` shell. You may have scripts which rely on the old format. To reformat `mongosh` output on a single line, use `EJSON.stringify()`.

Run a sample query in `mongosh` and `mongo` to see the different formats.

```
db.sales.find( { custId: 345 } )
```



Legacy output:

```
{ "_id" : ObjectId("64da90c1175f5091debcab26"), "custId" : 345, "purchaseDate" : ISODate("2023-07-04T00:00:00.000Z"), "quantity" : 4, "cost" : Decimal128("100.60") }
```

`mongosh` output:

```
db.sales.find( { custId: 345 } )
[
  {
    _id: ObjectId("64da90c1175f5091debcab26"),
    custId: 345,
    purchaseDate: ISODate("2023-07-04T00:00:00.000Z"),
    quantity: 4,
    cost: Decimal128("100.60")
  }
]
```

Reformat the output with `EJSON.stringify()`.

```
EJSON.stringify( db.sales.find( { custId: 345 } ).toArray() )
```



```
[{"_id":{"_id":"64da90c1175f5091debcab26"},"custId":345,"purchaseDate":ISODate("2023-07-04T00:00:00.000Z"),"quantity":4,"cost":{"cost":"100.60"}}
```

Learn More

- [EJSON](#) documentation
- Mozilla Developer Network `JSON.stringify()` documentation

